37. Design Distributed ID Generator

Key Terms:

Term	Full Name	What it does
Snowflake ID	Twitter's ID format	64-bit: timestamp + node ID + sequence
UUID	Universally Unique Identifier	128-bit random/time-based ID
UUID v7	Time-sortable UUID	New standard with timestamp prefix
Epoch	Reference timestamp	Starting point for timestamp bits
NTP	Network Time Protocol	Clock synchronization across servers
Clock Drift	Time desynchronization	Clocks getting out of sync

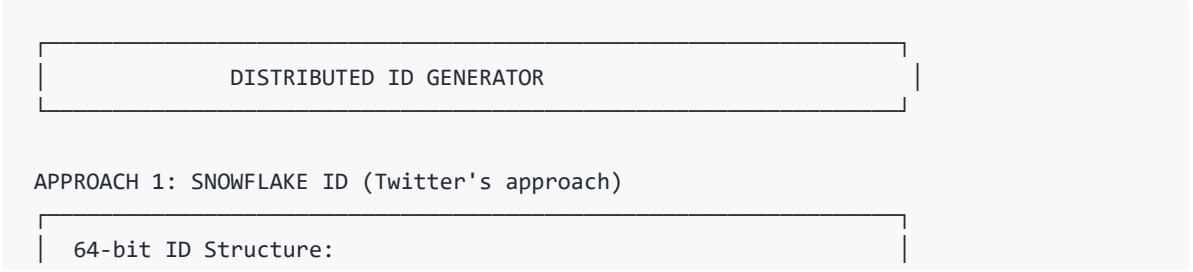
Problem Statement

Design a system that generates globally unique IDs at scale with properties like sortability, no coordination, and high availability.

Clarifying Questions to Ask

1. **Scale**: How many IDs per second? (100K+ per second?)
2. **Properties**: Unique, sortable, compact?
3. **Length**: Max ID length acceptable? (64-bit?)
4. **Coordination**: Can nodes coordinate, or must be independent?
5. **Ordering**: Strictly ordered or roughly time-ordered?
6. **Format**: Numeric, alphanumeric, or UUID?
7. **Failure modes**: What if clock drifts or node fails?

High-Level Design



```
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ 1 bit │ 41 bits      │ 10 bits  │ 12 bits              │   │
│  │ sign  │ timestamp    │ node ID  │ sequence             │   │
│  │ (0)   │ (millisec)   │          │ (per ms)             │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                 │
│  Capacity:                                                      │
│    - 41 bits timestamp: 69 years from epoch                     │
│    - 10 bits node: 1024 nodes                                   │
│    - 12 bits sequence: 4096 IDs per millisecond per node        │
│                                                                 │
│  Total: 4M IDs/second/node × 1024 nodes = 4B IDs/second         │
└─────────────────────────────────────────────────────────────────┘

APPROACH 2: UUID v4 (Random)
┌─────────────────────────────────────────────────────────────────┐
│  128-bit random ID: 550e8400-e29b-41d4-a716-446655440000        │
│                                                                 │
│  Pros: No coordination, truly unique                            │
│  Cons: Not sortable, 128 bits (large), poor index locality      │
└─────────────────────────────────────────────────────────────────┘

APPROACH 3: UUID v7 (Time-based, new standard)
┌─────────────────────────────────────────────────────────────────┐
│  128-bit with timestamp prefix: sortable + random               │
│                                                                 │
│  48 bits: Unix timestamp (milliseconds)                         │
│  4 bits:  Version (7)                                           │
│  12 bits: Random                                                │
│  62 bits: Random                                                │
│                                                                 │
│  Pros: Sortable, no coordination, standard format               │
│  Cons: 128 bits                                                 │
└─────────────────────────────────────────────────────────────────┘
```

API Design

ID Generation APIs

Method	Endpoint	Description
`POST`	`/api/v1/ids/generate`	Generate a single unique ID
`POST`	`/api/v1/ids/batch?count={n}`	Generate a batch of IDs
`GET`	`/api/v1/ids/{id}/decode`	Decode ID to extract timestamp, worker, sequence

Generate ID Response

```
POST /api/v1/ids/generate

Response: 200 OK
{
  "id": "175942649856712704",
  "id_hex": "0x02710000000001",
  "decoded": {
    "timestamp": 1672531200000,
    "datacenter_id": 1,
    "worker_id": 5,
    "sequence": 0
  }
}
```

Batch Generation

```
POST /api/v1/ids/batch?count=100

Response: 200 OK
{
  "ids": ["175942649856712704", "175942649856712705", ...],
  "count": 100,
  "generated_in_ms": 1
}
```

Worker Management APIs (Internal)

Method	Endpoint	Description
`POST`	`/api/v1/workers/register`	Register a worker node
`GET`	`/api/v1/workers`	List active workers and their IDs
`DELETE`	`/api/v1/workers/{workerId}`	Deregister a worker

Snowflake Implementation

```
public class SnowflakeIdGenerator {
    private static final long EPOCH = 1609459200000L; // 2021-01-01
    private static final long NODE_BITS = 10;
    private static final long SEQUENCE_BITS = 12;

    private static final long MAX_NODE_ID = (1L << NODE_BITS) - 1;
    private static final long MAX_SEQUENCE = (1L << SEQUENCE_BITS) - 1;
```

```
    private final long nodeId;
    private long lastTimestamp = -1L;
    private long sequence = 0L;

    public SnowflakeIdGenerator(long nodeId) {
        if (nodeId > MAX_NODE_ID) throw new IllegalArgumentException();
        this.nodeId = nodeId;
    }

    public synchronized long nextId() {
        long timestamp = System.currentTimeMillis();

        if (timestamp < lastTimestamp) {
            // Clock went backwards - wait or throw
            throw new RuntimeException("Clock moved backwards");
        }

        if (timestamp == lastTimestamp) {
            sequence = (sequence + 1) & MAX_SEQUENCE;
            if (sequence == 0) {
                // Sequence exhausted, wait for next millisecond
                timestamp = waitNextMillis(lastTimestamp);
            }
        } else {
            sequence = 0;
        }

        lastTimestamp = timestamp;

        return ((timestamp - EPOCH) << (NODE_BITS + SEQUENCE_BITS))
             | (nodeId << SEQUENCE_BITS)
             | sequence;
    }
}
```

Key Components

Component	Purpose	Technology
ID Service	Generate IDs	Stateless service
Node Registry	Assign node IDs	ZooKeeper / etcd
Clock Sync	Keep clocks aligned	NTP
Monitoring	Detect clock drift	Alerting

Scalability Considerations

- **No coordination**: Each node generates independently
- **Horizontal scaling**: Add nodes, assign unique node IDs
- **Clock sync**: Use NTP, monitor for drift > 1 second
- **Client-side generation**: Generate in application, no network hop
- **Batch pre-generation**: Pre-generate IDs for burst handling

Trade-offs

Decision	Option A	Option B	Recommendation
Format	UUID	Snowflake	**Snowflake** - smaller, sortable
Coordination	ZooKeeper	Config	**Config** - simpler for fixed nodes
Clock handling	Wait	Error	**Wait** - more available
Generation	Server	Client	**Client** - lower latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Raft Consensus	Node ID assignment via ZooKeeper/etcd	Tech Guide

---

How to Explain It Well

> **Opening**: "Distributed IDs need uniqueness without coordination. I'll use Snowflake: 64-bit IDs with timestamp, node ID, and sequence."

> **Key talking points**:
>
> - "Snowflake: 41 bits time + 10 bits node + 12 bits sequence"
> - "4096 IDs per millisecond per node, 1024 nodes = billions/sec"
> - "Time-ordered: IDs roughly sort by creation time"
> - "No coordination: each node generates independently"
> - "Clock drift: detect and wait, never go backwards"
